

Python Program

```
from flask import Flask, request, jsonify

app = Flask(__name__)

# User data
users = [
    {"id": 1, "name": "Tejaswini", "email": "tejaswini@gmail.com"},
    {"id": 2, "name": "Rahul", "email": "rahul@gmail.com"}
]

# GET - Display all users
@app.route('/users', methods=['GET'])
def get_users():
    return jsonify(users)

# POST - Add a new user
@app.route('/users', methods=['POST'])
def add_user():
    data = request.get_json()

    new_user = {
        "id": len(users) + 1,
        "name": data["name"],
        "email": data["email"]
    }

    users.append(new_user)
    return jsonify(new_user), 201

# PUT - Update user
@app.route('/users/<int:user_id>', methods=['PUT'])
def update_user(user_id):
    data = request.get_json()

    for user in users:
        if user["id"] == user_id:
            user["name"] = data.get("name", user["name"])
            user["email"] = data.get("email", user["email"])
            return jsonify(user)

    return jsonify({"message": "User not found"}), 404

# DELETE - Delete user
@app.route('/users/<int:user_id>', methods=['DELETE'])
def delete_user(user_id):
    for user in users:
        if user["id"] == user_id:
            users.remove(user)
            return jsonify({"message": "User deleted successfully"})

    return jsonify({"message": "User not found"}), 404

if __name__ == '__main__':
    app.run(debug=True)
```

After running:

```
python app.py
```

Output in terminal:

```
* Running on http://127.0.0.1:5000
* Debug mode: on
```

1. GET Request

Request:

```
GET http://127.0.0.1:5000/users
```

Output:

```
[
  {
    "email": "tejaswini@gmail.com",
    "id": 1,
    "name": "Tejaswini"
  },
  {
    "email": "rahul@gmail.com",
    "id": 2,
    "name": "Rahul"
  }
]
```

2. POST Request

Request:

```
POST http://127.0.0.1:5000/users
```

Input:

```
{
  "name": "Anjali",
  "email": "anjali@gmail.com"
}
```

Output:

```
{
  "email": "anjali@gmail.com",
  "id": 3,
  "name": "Anjali"
}
```

3. PUT Request

Request:

```
PUT http://127.0.0.1:5000/users/1
```

Input:

```
{
  "name": "Tejaswini Updated",
  "email": "updated@gmail.com"
}
```

Output:

```
{
  "email": "updated@gmail.com",
  "id": 1,
  "name": "Tejaswini Updated"
}
```

4. DELETE Request

Request:

```
DELETE http://127.0.0.1:5000/users/2
```

Output:

```
{
  "message": "User deleted successfully"
}
```

Result

The REST API was successfully created using Python Flask. It can create, read, update, and delete (CRUD) user data using RESTful API.